

✔ What is an Array in JavaScript?

An **Array** is an **ordered collection** of multiple values stored in a single variable.

Simple definition:

An array is like a container that holds multiple values (numbers, strings, objects, etc.) in a sequence, accessible by their index.

7.0 Array Methods

7.1 Slice

7.2 Splice

7.3 At

7.4 Map

7.5 Filter

7.0 Array Methods

Array methods in JavaScript are functions that can be called on arrays to perform various operations. They help in manipulating and processing array data efficiently.

main.js	Output
<pre>1 let colors = ["red", "green", "blue"]; 2 3 console.log(colors.length); 4 console.log(colors[1]); 5 6 colors.push("yellow"); 7 console.log(colors); 8 9 colors.pop(); 10 console.log(colors); 11</pre>	<pre>3 green ['red', 'green', 'blue', 'yellow'] ['red', 'green', 'blue'] === Code Execution Successful ===</pre>

[Click me](#) <--

7.1 slice()

✓ Syntax:

`array.slice(startIndex, endIndex);`

- `startIndex` → Kahan se slice start hoga (included)
- `endIndex` → Kahan tak slice hoga (excluded)

📄 Code:

```
let fruits = ["apple", "banana", "cherry", "mango", "orange"];
let slicedFruits = fruits.slice(1, 3);

console.log(slicedFruits);
console.log(fruits);
```

['banana', 'cherry']
['apple', 'banana', 'cherry', 'mango', 'orange']

=== Code Execution Successful ===

```
let fruits = ["apple", "banana", "cherry", "mango", "orange"];
let slicedFruits = fruits.slice(1, 4);

console.log(slicedFruits);
console.log(fruits);
```

📤 Output:

```
[ 'banana', 'cherry', 'mango' ]
[ 'apple', 'banana', 'cherry', 'mango', 'orange' ]
```

🔍 Explanation:

- `slice(1, 4)` → index 1 se start kiya ("banana") aur index 4 tak ka slice liya (but 4 exclude hota hai), to "orange" nahi aayega.
- Original array **same rehta hai**, koi change nahi hota.

7.2 splice()

✓ Syntax:

`array.splice(startIndex, deleteCount, item1, item2, ...);`

- `startIndex` → kaha se changes start karne hain
- `deleteCount` → kitne elements remove karne hain
- `item1, item2...` → agar kuch add karna ho to

```
1 let colors = ["red", "green", "blue", "yellow"];
2 let removed = colors.splice(1, 1, "purple", "pink");
3
4 console.log("Removed:", removed);
5 console.log("Updated:", colors);
6
```

Removed: ['green']
Updated: ['red', 'purple', 'pink', 'blue', 'yellow']

=== Code Execution Successful ===

📄 Code:

```
let colors = ["red", "green", "blue", "yellow"];
let removed = colors.splice(1, 2, "purple", "pink");

console.log("Removed:", removed);
console.log("Updated:", colors);
```

📤 Output:

```
Removed: [ 'green', 'blue' ]
Updated: [ 'red', 'purple', 'pink', 'yellow' ]
```

🔍 Explanation:

- `splice(1, 2, "purple", "pink")`:
 - index 1 se 2 items remove kiye → "green", "blue"
 - Unki jagah daala → "purple", "pink"
 - **Original array change ho gaya.**
-

🔗 7.3 at()

✓ Syntax:

```
array.at(index);
```

- `index` → jo element chahiye uska index (positive ya negative)
-

📄 Code:

```
let numbers = [10, 20, 30, 40, 50];

console.log(numbers.at(2)); // 3rd element
console.log(numbers.at(-1)); // Last element
```

📤 Output:

```
30
50
```

⇒ w/o Add ARRAY (METHOD)

main.js	Output
1 let arr = [100, 200, 300];	undefined
2	300
3 console.log(arr[-1]); // ❌ undefined	
4 console.log(arr.at(-1)); // ✅ 300	=== Code Execution Successful ===
5	

```

2=>
main.js
1 let arr = [100, 200, 300];
2
3 console.log(arr[1]); // ✖ undefined
4 console.log(arr.at(-1)); // ✔ 300
5
Output
200
300
=== Code E

```

```

H : W ?
main.js
1 let arr = [100, 200, 300];
2
3 console.log(arr.at[1]); // ✖ undefined
4 console.log(arr.at(-1)); // ✔ 300
5
Output
undefined
300
=== Code Execu

```

📖 Explanation:

- `at(2)` → 3rd element (index 2) → 300
- `at(-1)` → last element (negative index) → 300
- **Original array change nahi hota.**

Agar aage aur array methods (like `map()`, `filter()`, `reduce()`) chahiye ho Hinglish mein, bas bol do.

✓ Arrow Function in JavaScript

Code → [Click me](#)

📖 Kya hota hai?

Arrow function ek **shorter syntax** hai functions likhne ke liye. Ye **function keyword** ki jagah **=> (arrow)** ka use karta hai.

Zaroor! 🤖

Chalo **Arrow Functions in JavaScript** ko **Hinglish** mein samjhte hain — syntax, code, output, aur explanation ke saath.

✓ Arrow Function in JavaScript

📖 Kya hota hai?

Arrow function ek **shorter syntax** hai functions likhne ke liye. Ye **function keyword** ki jagah **=> (arrow)** ka use karta hai.

◆ Syntax

◆ Normal Function:

```
function add(a, b) {  
  return a + b;  
}
```

◆ Arrow Function:

```
const add = (a, b) => {  
  return a + b;  
};
```

◆ One-line Arrow Function (short form):

```
const add = (a, b) => a + b;
```

📄 Example Code with Output

1- <code>const greet = () => {</code>	hello World
2- <code> console.log("hello World ");</code>	
3 <code>};</code>	=== Code Executi
4	
5 <code>greet();</code>	
6	

```
const greet = () => {  
  return "Hello, World!";  
};
```

```
console.log(greet());
```

📄 Output:

Hello, World!

main.js	   Share 	Output
<pre> 1 greet = (a,b) => { 2 console.log("hello World "); 3 console.log("hello World ",(a+b)); 4 }; 5 a=2 6 b=9 7 greet(a,b); 8 </pre>		<pre> hello World hello World 11 === Code Execution </pre>

🔍 Explanation:

- `const greet = () => {...}`

Ye ek arrow function hai jisme koi parameter nahi hai.

- Function Hello, World! return karta hai.
- `console.log(greet())` us value ko print karta hai.

🔗 More Examples

main.js	   Share 	Output
<pre> 1 multiply = (x, y) => x * y; 2 3 console.log("Mul is "+multiply(3, 4)); 4 </pre>		<pre> Mul is 12 === Code Execution </pre>

✓ 1. With Parameters:

```
const multiply = (x, y) => x * y;

console.log(multiply(3, 4));
```

Output: 12

✓ 2. Single Parameter:

```
const square = n => n * n;

console.log(square(5));
```

Output: 25

✓ 3. Without Parameter:

```
const sayHi = () => "Hi!";
```

```
console.log(sayHi());
```

Output: Hi!

⚠ Arrow Function Ki Limitations

Feature	Arrow Function	Normal Function
this ka behaviour	✗ Alag hota hai (lexical)	✓ Apna context
Constructor (new)	✗ Nahi ban sakta	✓ Ban sakta hai

Simple For Loop →

main.js

 Share

Run

Output

```
1 numbers = [1, 2, 3, 4];
2
3 for (let i = 0; i < numbers.length; i++) {
4   console.log(numbers[i]);
5 }
6
7
```

1
2
3
4

=== Code

7.4 Map

Theory: The `map()` method creates a new array populated with the results of calling a provided function on every element in the calling array.

main.js

 Share

Run

Output

```
1 const numbers = [1, 2, 3, 4];
2 const doubled = [];
3
4 for (let i = 0; i < numbers.length; i++) {
5   doubled[i] = numbers[i] * 2;
6 }
7 console.log(doubled);
8
```

[2, 4, 6, 8]

=== Code Execution S

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Output: [2, 4, 6, 8]
```

for DIRECT USE →

Code o/p in NaN

<pre>1 const numbers = [1, 2, 3, 4]; 2 const doubled = (numbers * 2); 3 console.log(doubled); // Output: [2, 4, 6, 8]</pre>	NaN === Code Execution Su
---	---

7.5 Filter

Theory: The `filter()` method creates a new array with all elements that pass the test implemented by the provided function.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const evenNumbers = numbers.filter(num => num % 2 === 0);

console.log(evenNumbers); // Output: [2, 4]
```

7.6 Reduce

Theory: The `reduce()` method executes a reducer function on each element of the array, resulting in a single output value. It takes a callback function and an initial value.

Example:

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((accumulator, currentValue) =>
  accumulator + currentValue, 0);
console.log(sum); // Output: 10
```

*Logic

- **reduce ka kaam:**

Yeh array ke elements ko ek single value mein reduce karta hai, yahan hum elements ka sum nikaal rahe hain.

- **Parameters:**

`reduce` method ko ek callback function aur ek initial value pass hoti hai.

Callback Function Parameters:

- **accumulator:** Yeh previous iteration ka result ya initial value hota hai.
- **currentValue:** Yeh array ka current element hota hai.

- **Initial Value:**

Callback function ke pehle argument (`accumulator`) ki initial value set karne ke liye ek value pass ki jati hai. Yahan wo 0 hai.

Callback Function Workflow:

`reduce` har element par callback function ko run karta hai. Iska flow dekhte hain:

Iteration	accumulator (Initial Result)	currentValue (Current Array Element)	Result (accumulator + currentValue)
1	0	1	0 + 1 = 1
2	1	2	1 + 2 = 3
3	3	3	3 + 3 = 6
4	6	4	6 + 4 = 10

7.7 Find

Theory: The `find()` method returns the value of the first element in the array that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const found = numbers.find(num => num > 3);
console.log(found); // Output: 4
```

How `find` Works in Iterations:

`find` ek ek karke array ke elements ko check karega:

Iteration	Current Element (num)	Condition (num > 3)	Result
1	1	1 > 3 (False)	Skip
2	2	2 > 3 (False)	Skip
3	3	3 > 3 (False)	Skip
4	4	4 > 3 (True)	Return 4

O/p is 4.

7.8 FindIndex

Theory: The `findIndex()` method returns the index of the first element in the array that satisfies the provided testing function. If no elements satisfy the testing function, `-1` is returned.

Example:

javascript

```
const numbers = [1, 2, 3, 4, 5];  
const index = numbers.findIndex(num => num > 3);  
console.log(index); // Output: 3
```

logic→

How `findIndex` Works in Iterations:

Iteration	Current Element (num)	Condition (num > 3)	Action
1	1	1 > 3 (False)	Continue
2	2	2 > 3 (False)	Continue
3	3	3 > 3 (False)	Continue
4	4	4 > 3 (True)	Return index 3

- Jaise hi 4 (index 3) condition ko satisfy karta hai (`4 > 3`), `findIndex` wahan ruk jaata hai aur us element ka index (3) return karta hai.